



**B.Sc. (General/Special) Degree
Programme
Faculty of Applied Sciences
University of Sri Jayewardenepura**

ICT 1022 2.0 Computer System Architecture

Presented By:

Surani Tissera(PhD)

Department of Computer Science

Instruction Set Architecture



This is a property of Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.

Instruction

- Instruction Set Architecture (ISA)
 - Serves as an interface between software and hardware.
 - Typically consists of information regarding the programmer's view of the architecture (i.e. the registers, address and data buses, etc.).
 - Also consists of the instruction set.
- Many ISA's are not specific to a particular computer architecture.
 - They survive across generations.
 - Classic examples: IBM 360 series, Intel x86 series, etc.



Instruction set design issue

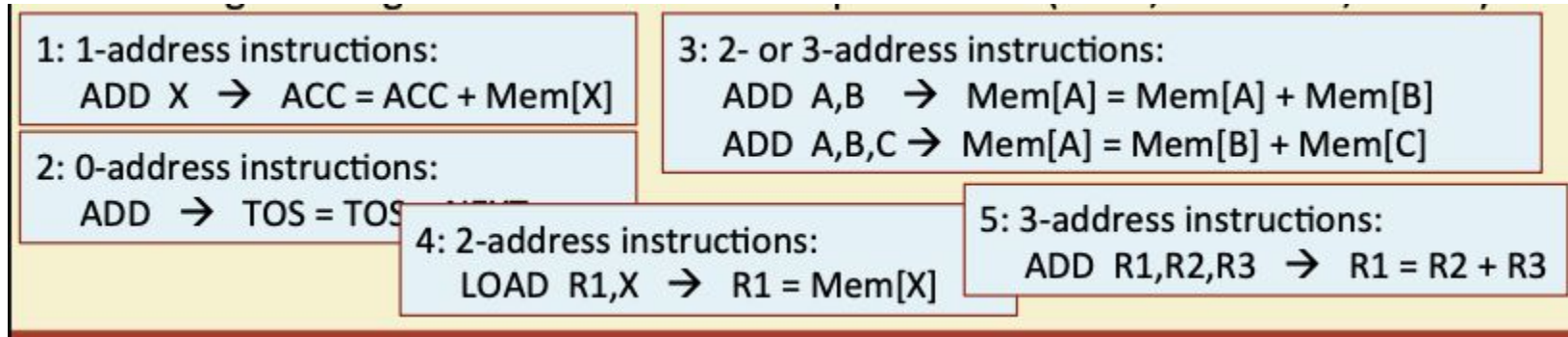
- Number of explicit operands:
 - 0,1,2 or 3.
- Location of the operands:
 - Registers, accumulator, memory, accumulator.
- Specification of operand locations:
 - Addressing modes: register, immediate, indirect, relative, etc.
- Sizes of operands supported:
 - Byte(8-bits), Half-word(16-bits), Word(32-bits), Double(64-bits), etc.
- Supported operations:



ADD, SUB, MUL, AND, OR, CMP, MOVE, JMP, etc.

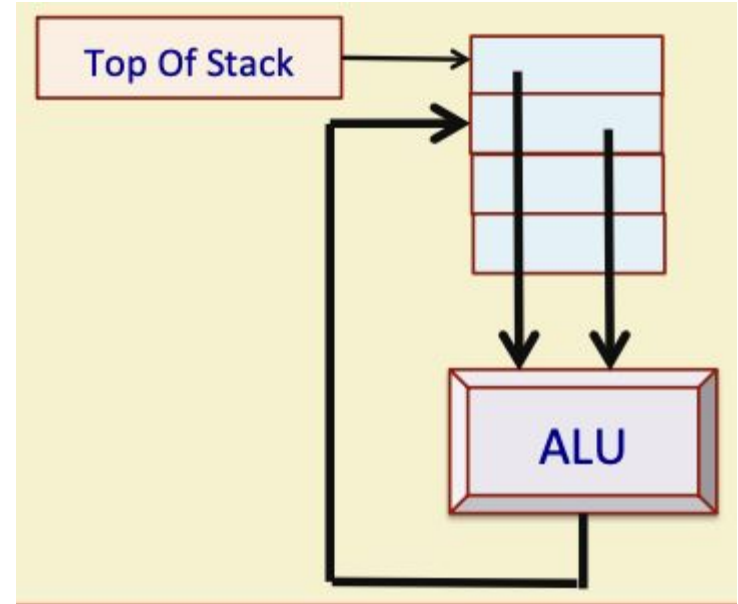
Evolution of Instruction Sets

1. Accumulator based: 1960's (EDSAC, IBM 1130)
2. Stack based: 1960-70 (Burroughs 5000)
3. Memory-Memory based: 1970-80 (IBM 360)
4. Register-Memory based: 1970-present (Intel x86)
5. Register-Register based: 1960-present (MIPS, CDC 6600, SPARC)



Example Code Sequence for $Z = X + Y$

- Stack machine:
PUSH X
PUSH Y
ADD
POP Z
- The ADD instruction pops two elements from stack, adds them, and pushes back result.



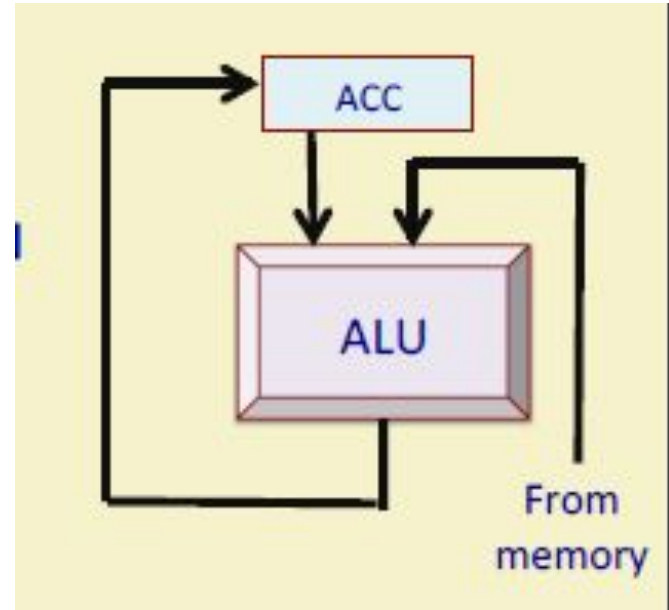
Accumulator based machine:

LOAD X // $ACC = Mem[X]$

ADD Y // $ACC = ACC + Mem[Y]$

STORE Z // $Mem[Z] = ACC$

- All instructions assume that one of the operands (and also the result) is in a special register called accumulator.



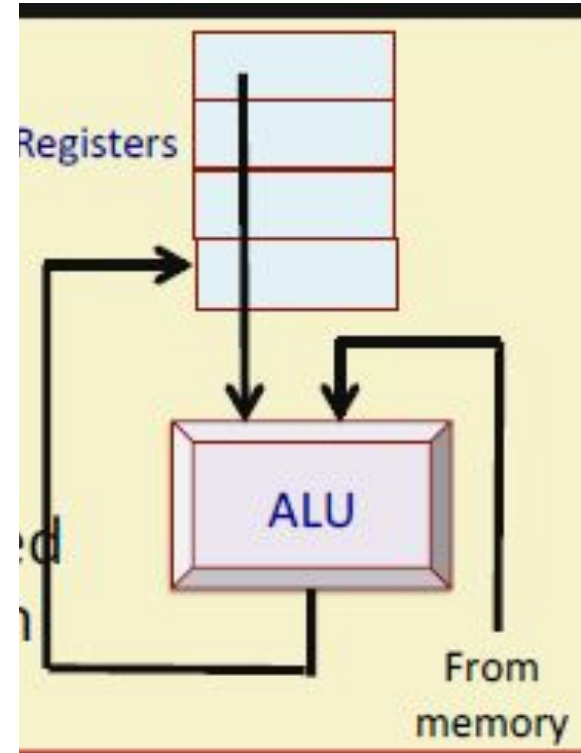
Register-Memory machine:

LOAD R2,X // $R2 = \text{Mem}[X]$

ADD R2,Y // $R2 = R2 + \text{Mem}[Y]$

STORE Z,R2 // $\text{Mem}[Z] = R2$

One of the operands is assumed to be in register and another in memory.



Register-Register machine:

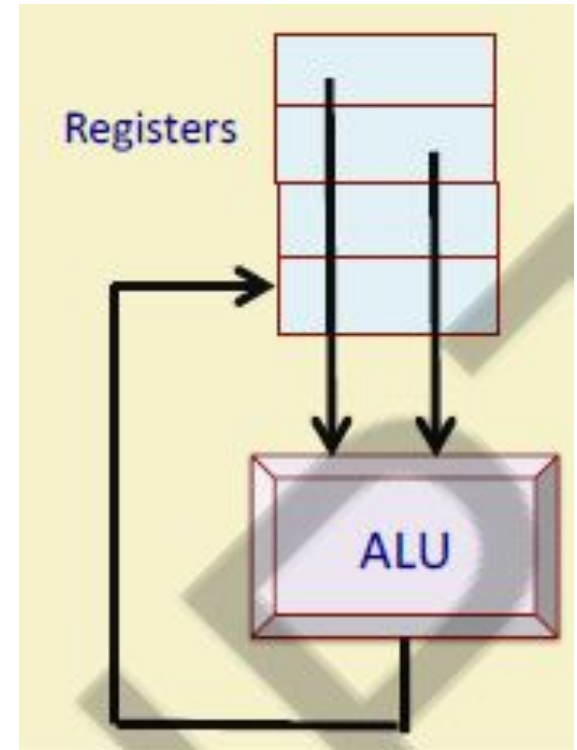
LOAD R1,X // $R1 = \text{Mem}[X]$

LOAD R2,Y // $R2 = \text{Mem}[Y]$

ADD R3,R1,R2 // $R3 = R1 + R2$

STORE Z,R3 // $\text{Mem}[Z] = R3$

- Also called load-store architecture, as only LOAD and STORE instructions can access memory.



About General Purpose Registers(GPRs)

- Older architectures had a large number of special purpose registers.
 - Program counter, stack pointer, index register, flag register, accumulator, etc.
- Newer architectures, in contrast, have a large number of GPRs.
- Why?
 - Easy for the compiler to assign some variables to registers.
 - Registers are much faster than memory.
 - More compact instruction encoding as fewer bits are required to specify registers.
 - Many processors have 32 or more GPR's.



Next

COMPARISON BETWEEN VARIOUS ARCHITECTURE TYPES

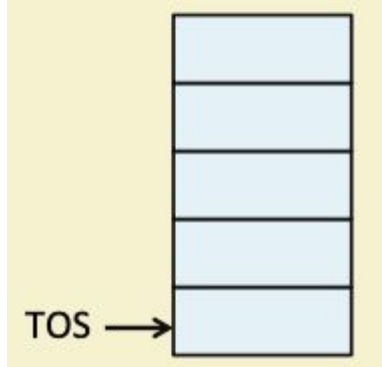


This is a property of Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.

(a) Stack Architecture

- Typical instructions:

PUSH X, POP X ADD, SUB, MUL, DIV



- Example: $Y = A/B - (A - C * B)$

PUSH A

PUSH B

DIV

PUSH A

PUSH C

PUSH B

MUL

SUB

SUB

POP Y



(b) Accumulator Architecture

Typical instructions:

LOAD X,

STORE X

ADDX, SUBX, MULX, DIVX

Example: $Y = A / B - (A - C * B)$

LOAD C

MUL B

STORE D // $D = C * B$

LOAD A

SUB D

STORE D // $D = A - C * B$

LOAD A

DIV B

SUB D

STORE Y



(c) Memory-Memory Architecture

Typical instructions (3 operands):

ADD X,Y,Z

SUB X,Y,Z

MUL X,Y,Z

Typical instructions (2 operands):

MOV X,Y

ADD X,Y SUB X,Y MUL X,Y

Example: $Y = A / B - (A - C * B)$

DIV D,A,B

MUL E,C,B

SUB E,A,E

SUB Y,D,E

MOV D,A

DIV D,B

MOV E,C

MUL E,B

SUB A,E

SUB D,A



d) Load-Store Architecture

- Typical instructions:

LOAD R1,X

STORE Y,R2

ADD R1,R2,R3

SUB R1,R2,R3

Example: $Y = A / B - (A - C * B)$

LOAD R1,A

LOAD R2,B

LOAD R3,C

DIV R4,R1,R2

MUL R5,R3,R2

SUB R5,R1,R5

SUB R4,R4,R5

STORE Y,R4



Registers: Pros and Cons

- The load-store architecture forms the basis of RISC ISA.
 - We shall explore one such RISC ISA, viz. MIPS.
- Helps in reducing memory traffic once the memory data are loaded into the registers.
- Compiler can generate very efficient code.
- Additional overhead for save/restore during procedure or interrupt calls and returns.
 - Many registers to save and restore.



Thank
you!

dreamstime

